

Terminal

Architecture, Data Sources & Open-Source Security Guide

Created by **Oscar Zarraga Perez**

A technical reference describing how the project collects, caches, and serves market data — written so the source code can be released publicly without exposing credentials or personal information.

Copyright © 2026 Oscar Zarraga Perez · All rights reserved

Document Version: **2.0**

Released without warranty. See next page.

Disclaimer & Limitation of Liability

No warranty. The Terminal project is provided “AS IS” and “AS AVAILABLE”, without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, accuracy, completeness, non-infringement, or continued availability of any upstream data source. The project is a personal-use, educational software experiment. Nothing in this document or in the source code creates a contract, an engagement, or a service-level commitment of any kind.

Not investment advice. The data displayed by the project is aggregated from third-party public sources whose accuracy, timeliness, and continuity cannot be guaranteed. The project is not a broker-dealer, investment adviser, financial planner, accountant, auditor, or tax adviser. None of the output of the project, of this document, or of the source code constitutes investment, legal, accounting, tax, or financial advice. Any decision to buy, sell, hold, or evaluate a security is the sole responsibility of the person making that decision and should be taken only after consulting an appropriately licensed professional.

Limitation of liability. To the maximum extent permitted by applicable law, the authors and contributors of the project disclaim all liability for any direct, indirect, incidental, special, consequential, exemplary, or punitive damages — including but not limited to lost profits, lost data, trading losses, missed opportunities, regulatory fines, reputational harm, or any other commercial damage — arising out of or in connection with the use, misuse, or inability to use the project, the documentation, the source code, or any data obtained through the project, even if advised of the possibility of such damages.

Operator responsibility. Anyone who downloads, runs, modifies, redistributes, or deploys the project (the “operator”) is solely responsible for: (a) compliance with the terms of service of every upstream data source the project queries; (b) compliance with all applicable laws, including securities, market-data, privacy, export, and tax laws in the operator's jurisdiction; (c) the security of any API keys, credentials, or other secrets the operator chooses to configure; (d) any consequences arising from public, commercial, or multi-user deployments of the project.

Third-party services. The project queries third-party endpoints. It does not control, endorse, or warrant any third-party content, service availability, accuracy, or terms. Endpoints may change, rate-limit, or cease without notice. Trademarks, service marks, and logos referenced in this document are the property of their respective owners; references are nominative and do not imply endorsement, affiliation, or sponsorship in either direction.

By using the project in any form, the operator acknowledges and agrees to all of the above. Operators who do not agree must not use, fork, or redistribute the project.

Authorship. Terminal was created and authored by **Oscar Zarraga Perez**. Copyright © 2026 Oscar Zarraga Perez. Released under the MIT License. Attribution to the author must be preserved in all copies and substantial portions of this software, including in source-file headers, in the rendered application footer, and in this disclaimer page. Removing or obscuring the attribution violates the MIT License under which this project is distributed.

Technical Documentation

Architecture, every data source, every endpoint, the in-app tab proxy, the cache and retry strategy, the garbage-data detector, and the full build history from two separate Phase-1 terminals into one unified application. Companion to SETUP.pdf (install + quick-start).

Document Version 2.0 — Copyright © 2026 Oscar Zarraga Perez · MIT License.

1. Architecture

The terminal is intentionally minimal in dependencies. The entire backend is one Python file (markets_data_api.py, ~8,900 lines) that uses only the standard library — no pip install required. It boots an HTTP server on port 8787 that does three things:

- Serve the static HTML pages (terminal.html shell + three sub-pages).
- Aggregate data from public APIs / RSS feeds / HTML scrapes and expose them as /api/* JSON endpoints.
- Proxy external pages through /api/proxy?url=... so SEC filings, press releases, and news articles can be embedded inside the terminal as new in-app tabs without spawning a Chrome window.

Page topology

terminal.html is the shell — a tab manager with a top tab bar. It pre-loads three pinned tabs as iframes pointing at the three sub-pages, and listens for postMessage events. When a sub-page wants to open an external link, it posts {type:'unified-terminal:open-tab', url, title} — the shell creates a new iframe whose src is /api/proxy?url=<target>.

Tab / page mapping:

Tab	Sub-page	Primary endpoints
Markets	markets.html	/api/quote, /api/chart, /api/summary, /api/news-all, /api/press, /api/sec/*
Watchlist Feed	calendar.html	/api/news-all/<sym>, /api/press/<sym>, /api/summary/<sym>, /api/earnings/rich-batch
Policy Watch	policy.html	/api/policy/today (fans out to 10 free sources in parallel)

Request flow for a single quote

Watchlist queries /api/quote?symbols=AAPL,MSFT,NVDA,GOOGL. The route classifies each ticker into one of three buckets based on its shape:

- **Stocks** — standard equity symbols. Yahoo v7 batch first, then Finnhub parallel as backup, then per-symbol shape_quote.
- **Non-stocks** — indices (^*), FX (=X), commodity futures (=F), DX-Y.NYB. Same Yahoo v7 batch but skip Finnhub/FMP (they do not cover these). Fall back to Yahoo v8 chart + Stooq via parallel

shape_quote.

- **Crypto** — everything `is_crypto()` recognizes. Always per-symbol `shape_quote`, never touches Yahoo — goes to CoinGecko.

2. Data sources

Every source the terminal uses, broken down by what it is for. All sources are free; the four key-protected providers in the bottom section are *optional* — the terminal works fully without any keys.

Free / no key required

Source	What it provides	Endpoint base
Yahoo Finance v7	Batch quotes for stocks/indices/FX/commodities	quotes1.finance.yahoo.com
Yahoo Finance v8	OHLCV charts, single quotes, meta	query1.finance.yahoo.com/v8
Yahoo HTML / RSS	News, press releases, SEC filings list	finance.yahoo.com
Stooq	CSV quotes + history fallback for ~everything	stooq.com/q/l/
CoinGecko	BTC + crypto price, market cap, dominance, history	api.coinbase.com/v3
mempool.space	Block height, hash rate, mempool, fees, Lightning	mempool.space/api/*
alternative.me	Bitcoin Fear & Greed Index	api.alternative.me/fng
SEC EDGAR	10-K, 10-Q, 8-K filings; XBRL facts; Form 4	data.sec.gov / www.sec.gov
Nasdaq public calendar	Earnings, economic events, IPOs, dividends	api.nasdaq.com/api/calendar/*
Nasdaq analyst data	Price targets, ratings, EPS forecast, surprises	api.nasdaq.com/api/analyst/*
Federal Register API	SEC/CFTC/Fed/Treasury rules + comments	fedregister.gov/api/v1
Federal Reserve RSS	FOMC, monetary policy, enforcement, speeches	federalreserve.gov/feeds
Treasury RSS	Treasury press, OFAC actions	home.treasury.gov
SEC press RSS	SEC press releases	sec.gov/news/pressreleases.rss
CFTC news XML	CFTC press releases, rule schedules	cftc.gov/news.xml
Senate Banking RSS	Banking Committee hearings, markups	banking.senate.gov
Senate Ag RSS	CFTC oversight hearings	agriculture.senate.gov
House Fin Services RSS	Hearing schedules, bill markups	financialservices.house.gov
House Ag RSS	Crypto-relevant Ag hearings	agriculture.house.gov
ECB press RSS	ECB decisions, Lagarde calendar	ecb.europa.eu
Bank of England news RSS	BoE rate decisions	bankofengland.co.uk

Source	What it provides	Endpoint base
Farside	Daily BTC + ETH spot ETF net flows	farside.co.uk/btc, /eth
Google News RSS	Symbol-keyed news aggregation	news.google.com/rss/search
MarketWatch RSS	Symbol-filtered news stream	feeds.content.dowjones.io
Finviz	Snapshot scrape (52w stats, earnings date)	finviz.com/quote.ashx
StockAnalysis.com	Fundamentals + intl ADR coverage	stockanalysis.com
Cboe	Option data fallback	cdn.cboe.com

Optional keyed providers (free tiers)

Provider	What it adds	Free quota
Finnhub	Real-time stock quotes; deep metrics (P/E, beta, 52w H/L, EPS, dividend, market cap, shares out); logo; earnings	60 req/min · 1000/day
Financial Modeling Prep	Wider equity universe inc. ADRs/intl; per-quote market cap; batch endpoint	250 req/day
Twelve Data	Batch quote endpoint; covers some intl tickers Yahoo does not	8 req/min · 800/day
Alpha Vantage	Optional cross-check for stocks/FX	5 req/min · 500/day

3. API endpoints

Markets / quotes / fundamentals

Endpoint	Returns
GET /api/healthz	Server health check
GET /api/keys/status	Which keyed providers are configured
GET /api/quote?symbols=A,B,C	Yahoo-shape batch quote; provider-routed by ticker type
GET /api/summary/<sym>;	Full quoteSummary: price, profile, financials, SEC filings, recommendations
GET /api/chart/<sym>;?range&interval	Yahoo-shape OHLCV chart payload
GET /api/financials/<sym>;	Income / balance / cash flow (annual + quarterly)
GET /api/profile-ext/<sym>;	Extended company profile (sector, industry, intl)
GET /api/statistics/<sym>;	Key statistics (P/E, beta, etc.)
GET /api/analyst/<sym>;	Price targets, ratings, EPS forecast, surprise history
GET /api/insider/<sym>;	SEC Form 4 insider transactions
GET /api/holders/<sym>;	Top institutional holders (Nasdaq)
GET /api/dividends/<sym>;	Dividend history
GET /api/earningsdate/<sym>;	Next earnings date + consensus EPS + annual forecast
GET /api/earnings/rich/<sym>;	Combined: next date, period, EPS est, history, market cap, logo
GET /api/earnings/rich-batch?symbols=...	Batch version of above, parallel server-side
GET /api/options/<sym>;	Yahoo-shape options chain

News / press releases / filings

Endpoint	Returns
GET /api/news-all/<sym>;	Aggregated news (Google News + Yahoo + MarketWatch + Fnnhub + FMP), dedupe
GET /api/press/<sym>;	Press releases (SEC 8-K 99.1, BusinessWire, GlobeNewswire, PRN, Nasdaq, Yahoo
GET /api/sec/filings/<sym>;	SEC EDGAR filing list with direct links
GET /api/sec/insider/<sym>;	Form 4 insider transactions

Endpoint	Returns
GET /api/sec/facts/<sym>;	Company-facts XBRL

Calendar (Nasdaq public API)

Endpoint	Returns
GET /api/calendar/earnings?date=YYYY-MM-DD	Earnings reports scheduled on that date
GET /api/calendar/economic?date=YYYY-MM-DD	Economic events (CPI, NFP, FOMC, etc.)
GET /api/calendar/dividends?date=YYYY-MM-DD	Ex-dividends on that date
GET /api/calendar/splits?date=YYYY-MM-DD	Stock splits on that date
GET /api/calendar/ipos?date=YYYY-MM	IPOs for the month (priced + filed + upcoming)

Policy Watch (free aggregators)

Endpoint	Returns
GET /api/policy/today	Aggregated payload: 10 sources fanned out in parallel
GET /api/policy/fed-register?back=N&forward=N	Federal Register documents, comment deadlines
GET /api/policy/fed	Federal Reserve press RSS
GET /api/policy/sec-press	SEC pressreleases.rss
GET /api/policy/cftc	CFTC news.xml
GET /api/policy/treasury	Treasury press feed
GET /api/policy/committees	Senate Banking + Senate Ag + House Fin Services + House Ag
GET /api/policy/etf-flows?asset=btc eth	Farside daily ETF net flows scrape
GET /api/policy/ecb	ECB press RSS
GET /api/policy/boe	Bank of England news RSS
GET /api/policy/links	The 54-link curated directory

In-app tab proxy

Endpoint	Returns
GET /api/proxy?url=...	Fetches external URL server-side, strips X-Frame-Options, injects base href + CSS r

4. How the in-app tab proxy works

When the user clicks an SEC filing or a news article inside the terminal, we do not want a Chrome window to pop. Instead, the click is intercepted and routed through `/api/proxy?url=<target>`, which:

- **Picks request headers** based on the target host. SEC.gov demands a contact email in the User-Agent; everything else gets a standard browser UA with a Referer.
- **Performs the GET** with urllib, follows redirects, captures the final URL.
- **Rewrites the HTML**: injects `<base href>` from the final URL so relative CSS/images resolve; strips inline `<meta http-equiv="content-security-policy">`, `x-frame-options`, and refresh tags; injects a CSS reset so the page renders on white even if its stylesheet fails inside the iframe; injects a click-interceptor that catches link clicks and form submits inside the proxied page and pops them as new in-app tabs too.
- **Sends back** with `X-Frame-Options: ALLOWALL` and `Content-Security-Policy: frame-ancestors *` so the parent shell can embed it.

5. Reliability layers

Provider separation

The watchlist (Markets) and the Policy Watch / Watchlist Feed tabs are deliberately routed through different primary providers so they cannot drain each other's rate-limit budgets. Markets primary: Yahoo v7 batch → Finnhub backup. Watchlist Feed primary: news/press aggregators (mostly RSS). Policy Watch primary: Federal Register API + agency RSS — nothing shared with the Markets path.

Caches

Module-level cache (`_cache_get` / `_cache_put`) with per-key TTL. Used by every helper that hits the network. TTLs range from 10 seconds for stock quotes up to 24 hours for SEC company facts.

Per-source soft cache for the Bitcoin Intel endpoint and the Policy Watch helpers. Each source has its own TTL. The cache returns the last known good value even if a refresh fetch fails — it never overwrites real data with empty data.

Watchlist row cache keyed by sorted symbol-set. On each `/api/quote` call, if the fresh row for a ticker has no price (or differs from the cached price by more than 50% in 30 seconds — a clear sign of Yahoo degraded mode), the cached row is preferred.

Garbage-data detector

When Yahoo's v7 batch endpoint is overloaded, it returns 200 OK responses where many unrelated tickers share the same fake price. Inside `yahoo_v7_quotes_batch` there is a detector that scans the response after parsing: if more than 50% of the returned rows share the same `regularMarketPrice`, the whole batch is rejected with a stderr log and the caller falls through to the alternative provider chain.

Stale-while-error

Every per-source fetcher in the Bitcoin Intel and Policy Watch paths is wrapped in `_src_cached_or_fetch(name, fetcher)`. Logic: cache hit → return; cache miss → call fetcher; fetcher exception or empty return → serve last known good value. Net effect: prices and stats never blank out — they go stale and stay visible.

Concurrency / parallel fan-out

Where many independent network calls are needed (BTC Intel sub-fetches, Policy Watch aggregator, basket quotes, non-stock side panels, rich-earnings batch), the code uses `ThreadPoolExecutor(max_workers=8)`. Wall-time becomes `max(call time)`, not sum.

6. Build history (Phase 1 → Phase 2)

The unified terminal evolved from fusing two separate Phase-1 codebases:

- **Markets Data Terminal (Reuse-Pack)** — a single Python file plus single HTML page providing a full equity terminal: quotes, financials, news, press releases, SEC filings, options, insider transactions. Public-domain Unlicense.
- **Bitcoin Intelligence Terminal** — an Electron + React + TypeScript app dedicated to Bitcoin price, mempool/chain health, fear-greed, BTC treasury equity baskets, and a multi-asset performance comparison chart.

Major Phase-2 milestones

Milestone	What it added
Unified shell (terminal.html)	Top tab bar + iframe stack + postMessage routing
In-app tab proxy	/api/proxy with frame-header stripping + base href rewriting
SEC proxy fix	SEC-specific User-Agent + CSS reset injection so proxied SEC pages render
Yahoo garbage detector	Rejects responses where >50% of returned tickers share a single price
Finnhub primary for watchlist	Yahoo demoted to backup; fh_quotes_batch_parallel + fh_metrics enrichment
fh_metrics helper	Pulls /stock/metric to populate marketCap, P/E, beta, EPS, 52w H/L, divYield
Non-stock routing	Indices, FX, =F commodities, DX-Y.NYB skip Finnhub/FMP entirely
Editable side panels	Indices / FX / Crypto add+remove with localStorage persistence
Watchlist Feed tab	Per-ticker news/press/filings/earnings aggregator
Rich earnings + TradingView table	rich_earnings combines Finnhub history + Nasdaq next-date + Finviz time-of-day
Policy Watch tab	Federal Register API + 4 committee RSS + Fed/SEC/CFTC/Treasury press + ECB/BoE + Farside
Farside scraper rewrite	html.parser based, multi-URL fallback, parens-to-negative parsing
Stale-while-error policy	Last known good values served when upstream fails
Provider separation	Markets and Policy/Feed never share a primary rate-limit budget

7. File structure

Everything required to deploy is in the project folder. The Python file is the only non-static piece; the HTML files are vanilla and have no build step:

File	Purpose
markets_data_api.py	Backend HTTP server. ~8,900 lines, stdlib only. Boots on port 8787.
terminal.html	Shell page. Tab manager + postMessage bus. Served at /
markets.html	Markets tab UI. Patched Reuse-Pack terminal.
calendar.html	Watchlist Feed tab UI.
policy.html	Policy Watch tab UI.
bitcoin.html	Legacy Bitcoin Intel page (still served).
start_mac_linux.command	Mac / Linux launcher (bash).
start_windows.bat	Windows launcher (cmd).
keys.json.example	Template for the optional API keys file.
LICENSE	MIT, Copyright © 2026 Oscar Zarraga Perez.
README.md	Project overview.
CHANGELOG.md	Phase 2 changelog.
Terminal_Architecture_v2.pdf	Cover + disclaimer + this technical document (combined).
SETUP.pdf	Step-by-step setup guide.
.gitignore	Excludes keys.json, __pycache__, .env, IDE folders.

8. License & attribution

Released under the MIT License. Copyright © 2026 Oscar Zarraga Perez. The Phase-1 base codebases (Markets Data Reuse Pack and Bitcoin Intelligence Terminal) were public-domain Unlicense. Free / public data sources retain their own copyrights; this terminal does not redistribute their data, only fetches and displays it on the user's behalf.